Project overview

We are creating a game called Natural Selection to help players of rogue-like games with enemy variety burnout, through merging traditional AI decision trees and neural networks. We will create a neural network that can adjust to the player's playstyle to target their weaknesses, add more challenges to the game, and push them to improve on their weaknesses.

Scope

In scope

- Neural network incorporation to make enemies smarter over time (NN-01)
    - This is included because part of the very premise of our game is for the game to be able to adapt to the player over time. And furthermore, neural networks are advantageous for complex learning tasks and for forming an underlying model to describe a situation. Because a player's psychological properties could be thought of as an "underlying" cause that drives their externally measurable behavior, a neural net would then be especially useful if we do implement targeting of the player's psychological weaknesses.
- Runs on Mac, Windows, or Linux operating system with semi-modern hardware (OP-02)
    - These are three major operating systems that should cover a wide userbase, while still being limited to PC gameplay so that we do not have to adapt the game to a whole different interface.
- Controllable player character (CC-03)
    - This animated player character requirement means they can move in any 2d vector direction and trigger at least one attack through user input. This is necessary for the player to interact with our game.
- Progression system for players (PS-04)
    - A progression system is included to provide direction and purpose to players, giving them milestones to look forward to. It will include mechanics like leveling up, unlocking new levels, and gradually increasing difficulty to validate time investment and keep the player engaged for a longer period of time.
- Fully functional player vs enemy combat system (CS-05)
    - This requirement is needed because we will have the enemy AIs training to improve metrics based on the combat system. The system will include health for the player and enemies, unique attacks for the player and enemies, and on-death resource cleanup/reuse.

Under evaluation

- Procedurally generated levels (PG-06)
    - Procedurally generated levels are a core part of roguelikes and ensure that every run is unique. It also forces players to react dynamically to whatever they encounter in each run, which ties in to the way enemies will also be doing the same with the player. In addition, procedurally generating each level allows for deep and complex maps without having to manually design every single room by hand.

- Audio assets (SFX/soundtrack) (AA-07)
    - Audio assets include sound effects and background music. These assets would highly enhance the experience of playing our game, but they are outside the scope of software development in general. Including audio assets is difficult due to time constraints, so if we have extra time towards the end of the project, we will implement audio assets.
- Focus adaptation on targeting psychological weaknesses to push the player into improving (PW-08)
    - This could be an interesting approach to adaptation to the player's playstyle, though it is still under evaluation due to its potential complexity and/or difficulty in testing and verifying it. It could be one way of addressing the challenge of having the game adapt to the player without it taking on what feels like "meaningless" difficulty. If the player knows that the game is trying to push them towards specific forms of growth, that places a reason behind the difficulty, and gives the player a direction.
    - To determine whether it is in scope, we will begin by attempting to create a core functional version of the product without it, and seeing whether we still have time. Then, we will see whether we are able to successfully implement it in the remaining time.

Out of scope

- Co-op/multiplayer (CM-09)
    - Co-op and multiplayer are out of scope due to feasibility constraints. Making a singleplayer roguelike with neural network integration is already difficult enough, there's no need for us to figure out how to add multiplayer functionality on top of that. It wouldn't fit the goal of our game either and would probably take much too long to actually implement.
- 3D visuals (3D-10)
    - 3D visuals are out of scope for our project due to feasibility constraints. Nobody on the team is a 3D modeler, so developing 3D assets and programming for 3 dimensions would overcomplicate the project. A 2D top down representation is much simpler to create and also
- Cross-platform development (CD-11)
    - Godot doesn't have native support for SDKs on platforms other than PC, so developing tools to develop for other platforms would be unfeasible and mostly unrelated to the project.

Assumptions

- Users are interested in the concept of the game
    - Users would probably have to be interested in what we're trying to do with our game to actually give it a shot. It wouldn't be very fun for someone who doesn't care about our game to play it.
- User has semi-modern hardware to run the game on
    - Developing the game to run on every kind of hardware out there would be highly complicated and way beyond the scope of the project. It is easier to simply just

keep our game limited to the bounds of semi-modern hardware rather than have to go in and adjust our game's performance to run on older hardware.

- Developing for PC exclusively
    - Adapting the game to console or mobile could require changing a lot of factors about how the game runs and would add a degree of complexity that we are considering as beyond the scope of this project.

Constraints

- It must be feasible to complete the project within a semester and a half.
- The project cannot require spending more than $450.

Requirements elicitation & prioritization

Evidence of our brainstorming process, and a reflection paragraph, are attached in another document in the Brightspace submission.

Must have:

- Controllable player character
- Neural networks incorporated in enemy AI/ dynamic difficulty
- Functional combat system
- Runs on any operating system with semi-modern hardware
- Progression system for players
- Some sort of tutorial
- Camera that follows player
- Map and player assets/textures
- Player + enemy animations

Should have:

- Targeting psychological weaknesses to encourage growth
- Procedural level generation
- Varied character/enemy movesets
- Intuitive and functional GUI
- Boss battles
- Narrative
- Randomized rewards per level, non-persistent in future runs
- Audio assets (sfx/ soundtrack)
- Controller support

Could have:

- High-effort assets (visual character design, etc.)
- Aesthetically pleasing GUI
- Different themed levels
- Dialog system for characters
- Multiple weapons/items
- Live updates
- Personal leaderboard/PB
- Real-time enemy behavior adjustment (rather than between rounds)

- Local co-op

Won't have:

- Multiplayer
- 3D visuals
- Open world
- One-time completion
- Microtransactions

Software requirements

1	SRID: NN-01 ID: FR-101 Title: Neural Net Control of Enemies Statement: Behavior of the in-game enemies shall be controlled, either directly or indirectly, by the outputs of a neural net that takes as input various parameters about the player's gameplay. Rationale: This is a part of the basic premise of our game. The enemies must adapt to player behavior, and we are choosing to use a neural net in order to accomplish this, firstly because player behavior is something complex that it would be difficult to make more "manual" calculations about, and secondly because the ability of neural nets to form an implicit "model" of a situation could be useful if we take the approach of predicting the player's psychological weaknesses. Test method: While the game is running, and debug information is open in Godot, it can be observed that enemy actions are coming from a neural net that takes as input the desired parameters. Supporting context: Depending on an enemy's "strategy", it may react differently to the same information, for instance information about where the player is currently located. The neural net will be set up such that the outputs are or can be converted to a vote for or influence from certain predefined "strategies", in order to best respond to the player's playstyle. Tracing information: (SYS-01) "Neural networks should be incorporated into the enemy AI." Priority: Must Have
2	SRID: NN-01 ID: FR-102 Title: Neural Net Knowledge Persistence

	Statement: Upon player death, the enemy neural nets shall retain what they have learned from the player, in order to be better-adjusted to the player in the next playthrough. Rationale: If the neural nets were to reset upon player death, this would make the interval in which the neural nets can learn even shorter than it would otherwise be. If they retain their knowledge, they have more data from which to learn and can more effectively adapt to the player. Test method: There can be a direct means of testing, where the values in the neural nets are inspected as the game runs and it is verified that they are retained upon player death. Supporting context: The collection of enemy neural nets upon player death will be considered as a generation in the neuroevolution process. Data in them being retained may therefore mean that the current weights are used to create the next generation, which is then what the player sees, rather than the same generation again. Tracing information: (SYS-02) “There should be data persistence for neural network and player progression.” Priority: Must Have
3	SRID: NN-01 ID: FR-103 Title: Enemy Learning Through Neuroevolution Statement: When the player advances into a new room/level within the game, a new generation of enemies will be formed via neuroevolution from the best-performing enemies (by a metric that we determine) of the previous generation. Rationale: Neuroevolution is a more effective way for the neural nets to learn in this context, than for them to learn through other methods such as supervised learning. As every player is different, it would be quite difficult to create a training set of adequate size for supervised learning. Neuroevolution, with its evolution-like mechanisms, can allow for quick adaptation directly off of player gameplay. Test method: While testing, we can log or display in-game the fitness score of each enemy, and observe whether the next generation appears more like the top performers of the previous generation. Supporting context: Neuroevolution is one way of training a neural net, where a collection of different versions of the neural net with randomized weights are created, and a performance metric is defined. Whichever nets perform best on the metric are able to “clone”, “reproduce”, and/or “mutate” to form the next generation. This process is then repeated until the task is “learned.” Tracing information: (SYS-01) “Neural networks should be incorporated into the enemy

	AI.” Priority: Must Have
4	SRID: PW-08 ID: FR-104 Title: Explicit Prediction of Psychological Traits Statement: Each enemy shall in some component in its neural net structure output an explicit prediction(s) about the player’s psychological traits. Rationale: Having the models make explicit predictions about psychological traits guarantees that there is at least an attempt to take them into account. If prediction were to instead go straight from input to enemy behavior output, there could be a whole number of other factors that the models are truly “learning” from, and it would be near-impossible to say whether any implicit psychological model of the player has been formed. Test method: The neural net structure can be examined, and there should be a clearly visible intermediate output stage that contains psychological predictions about the player (values from 0 to 1 for instance predicting likelihood of the player having certain traits.) Supporting context: There will either be additional neural network layers, or some kind of deterministic system, to transfer the psychological predictions to the form of the final output (enemy strategies.) Tracing information: (SYS-08) “The game should push the player towards overcoming certain psychological weaknesses through targeting them through enemy adaptation.” Priority: Should Have
5	SRID: PW-08 ID: FR-105 Title: Visibility of Enemy Strategy Statement: When an enemy attempts to target a psychological weakness or adapts in any other way, information about its strategy shall be accessible to the player through the menu or through hovering over the enemy with a cursor. Rationale: This allows the player to have a clearer idea, if they desire, of what direction the game is attempting to push them in. This makes it easier for them to assign a meaning to the challenge of the game. Test method: This can be observed through gameplay, through advancing in the game and checking if the desired information is available in the intended location (the game menu or by hovering over an enemy with the cursor.) The information that is displayed

	can also be double checked against actual states in the code. Supporting context: There will be a pre-specified collection of different weaknesses that the game can try to detect, and/or strategies that the enemies can take on in varying degrees. It can thus be reported to what degree a particular enemy is drawing from each of these strategies. Tracing information: (SYS-08) “The game should push the player towards overcoming certain psychological weaknesses through targeting them through enemy adaptation.” Priority: Should Have
6	SRID: CC-03 ID: FR-106 Title: Player Movement Controller Statement: While the user is inputting a movement direction and there is no collision object blocking the player character’s path, the Player Movement Controller shall move the player in the specified direction at the player’s predefined speed. Rationale: Allowing the user to control their player character is the primary method through which the user interacts with our project. It is an essential feature for our project to be functional. Test method: Create a test environment with collision edge cases. Run 10 tests in the test environment. Verify that the collision and movement work together without bugs. Spawn player character inside a collision object to simulate a collision error to test behavior. Log any bugs. Supporting context: Collision objects are usually environment features that the player character cannot overlap with. Test environment is a basic map made of collision objects in unusual orientations to test edge cases. Tracing information: (SYS-05) “There should be functional player controls, with an engaging character moveset.” Priority: Must Have
7	SRID: CC-03 ID: FR-107 Title: Player State Machine Statement: When a player state transition trigger occurs, the Player State Machine shall exit the current player state, enter the new player state, and hand over player control logic to that state until the next state transition occurs.

	Rationale: Having a state machine to control the player character's different state is necessary to avoid over complicating the logic to handle the user's input during different player character actions and animations such as attacking or idling. Test method: Create a suit of integration tests to simulate all user input that would lead to full traversal of the state machine graph. Observe for any errors in the output logic. Supporting context: The basic array of states the player must have include idle, walk, attack, and death. Each of these states have different logic to control how the player character behaves. For instance, attempting to move the player character in the attack state should not play the walking animation like it would in the walk state. The number of states the player has may expand as required by other systems in the project. Tracing information: (SYS-05) "There should be functional player controls, with an engaging character moveset." Priority: Must Have
8	SRID: CS-05 ID: FR-108 Title: Melee Attack Collision System Statement: When a player or enemy performs a melee attack, the Melee Attack Collision System shall enable a collision hurtbox for the duration of the attack that only checks for collisions with the player character or enemies. When the collision hurtbox detects a collision, the Melee Attack Collision System shall decrement affected player or enemies health by the attack power or trigger their death state, if their health drops below 0. Rationale: A melee attack system is necessary for combat to be functional in the project. Furthermore, damage will be one of the primary data points used to train our reinforcement learning model controlling the enemy AI. Test method: Build a test level with static enemies that don't move and debug labels showing player health and states. Use this level to test hurtbox collision and health decrementing. Supporting context: A collision hurtbox is a collision detection that only checks for collision with specified collision objects like a player or enemy instead of the environment. Log any bugs. Tracing information: (SYS-03) "There should be a fully realized combat system against enemies." Priority: Must Have
9	SRID: CS-05 ID: FR-109

	Title: Projectile Attack System Statement: When a player or enemy performs a ranged projectile attack, the Projectile Attack System shall spawn a projectile entity with a collision hurtbox for the duration of the attack. When the collision hurtbox detects a collision, the Projectile Attack Collision System shall decrement affected player or enemies health by the attack power or trigger their death state, if their health drops below 0. Rationale: A projectile attack system is good for combat to be varied. Furthermore, damage will be one of the primary data points used to train our reinforcement learning model controlling the enemy AI. Test method: Build a test level with static enemies that don't move and debug labels showing player health and states. Use this level to test hurtbox collision and health decrementing. Log any Bugs Supporting context: Projectiles vary from melee attack in how they are handled. Projectiles must be separate spawned entities with a timed lifetime. Melle attacks are inherently attached to the player or enemy performing the attack. Tracing information: (SYS-03) "There should be a fully realized combat system against enemies." Priority: Should Have
10	SRID: AA-07 ID: FR-110 Title: Soundtrack Player System Statement: When a soundtrack change signal is emitted, the Soundtrack Player System shall cause the background soundtrack to change to the specified song with any additional configurations sent by the signal. Rationale: The ability to place soundtrack triggers at certain points or thresholds in levels to change the background music will make much easier to implement theming around a level without Test method: Create a test level with every variation of soundtrack signal trigger. Use the level to run through each trigger. Log any bugs. Supporting context: A signal in Godot is a bit like a hardware interrupt. Emitting a signal allows any Node listening for that signal to trigger a function call upon receiving it. In this case the Node listening for signals would be the Soundtrack Player System. Tracing information: (SYS-06) "There should be an intuitive and aesthetically fitting GUI, and a functional soundtrack."

	Priority: Should Have
11	SRID: PS-04 ID: FR-111 Title: Experience & Level Tracking System Statement: The system shall track numerical player attributes like experience points (XP), current character level, and stat modifiers (health, attack, defense). Rationale: Having such a system quantifies player growth, structures pacing and growth, and delivers a return on time and effort that helps increase player retention. Test method: Create a test environment with simulated player attributes. Print the system outputs to the console and then observe how those outputs change when altering specific player attributes. Adjust the system depending on how well it performs and ensure that every output matches what is expected from the test values. Supporting context: Experience points are game mechanics that allow for a player's time and effort to be quantified in a reward system. Gaining enough experience points (XP) will cause a player to level up, commonly resulting in stat increases and unlocking new abilities. Tracing information: (SYS-02) "There should be data persistence for neural network and player progression." Priority: Could Have
12	SRID: PS-04 ID: FR-112 Title: Exponential Leveling Thresholds Statement: This system shall calculate level thresholds dynamically using mathematical formulas that exponentially increase based on the player level rather than having a hard-coded amount for each one. Rationale: Incentivizes players with easy early levels to progress further in their run and slows down progression proportionally as the run continues. Creates reward loops that trigger a sense of accomplishment in the player. Test method: Create a test environment with simulated level thresholds. Print the system outputs to the console and then observe how the amount of XP required for each level increases over time. Adjust the system if needed to ensure an appropriate amount of growth for each level threshold.
13	SRID: PS-04

	ID: FR-113 Title: Player Data Persistence Statement: After each run, the progress achieved by the player must be saved locally to provide the player with a detailed look at their personal statistics for that run. Those stats will include metrics like time spent, enemies killed, and levels cleared. Rationale: Players can look back at their past progress and use that to plan ahead for future runs, providing them with a way to visualize their own growth. Test method: Build a rudimentary player data saving system with a basic proof-of-concept game level with the objective of collecting some object (like snake). Then, run through the game once and check to see if the data on how many objects collected is saved to the local device. Supporting context: Our game is a roguelike, so there won't be any sort of metaprogression through mechanics like permanent upgrades or item unlocks like in other games. The only way for players to view how well they did in past runs will be through their saved player data. Tracing information: (SYS-02) "There should be data persistence for neural network and player progression." Priority: Must Have
14	SRID: PG-06 ID: FR-114 Title: Procedural Generation Algorithms Statement: These algorithms shall procedurally generate different types of level designs depending on the player's progression during their run. There will be two main types of levels, dungeon and hub, each with their own unique features. Rationale: Procedurally generating levels allows for endless replayability and variety, unique challenges in each run, and increased developmental efficiency. Test method: Generate basic versions of dungeon and hub levels. Verify that they generate as intended and adjust the generation algorithms if needed. Log any major errors or bugs in the code. Supporting context: We already have two procedural generation algorithms, one that generates a large mass with a wide-open interior (hub) and another that generates more traditional dungeons with interconnected rooms and areas for secondary objectives. Tracing information: (SYS-04) "There should be procedurally generated levels." Priority: Should Have

15	SRID: PG-06 ID: FR-115 Title: Level Overview System Statement: This system shall be in charge of organizing and determining when a specific level is generated throughout each run. Rationale: Maintains a balance between difficulty and randomness, especially during the later stages of a run. Increasing difficulty will be balanced out by hub levels, which will periodically appear throughout each run and give the player a respite between dungeons. Test method: Generate multiple levels in sequence, with a particular focus on the frequency of both level types appearing. Dungeons should appear much more often than hubs, which should only appear after a minimum of 10 dungeons have been cleared. Supporting context: Assume that players will encounter gradual difficulty increases due to enemy AI adaptation. Tracing information: (SYS-04) “There should be procedurally generated levels.” Priority: Should Have
15	SRID: PS-04 ID: FR-115 Title: Exponential Leveling Thresholds Statement: This system shall calculate level thresholds dynamically using mathematical formulas that exponentially increase based on the player level rather than having a hard-coded amount for each one. Rationale: Incentivizes players with easy early levels to progress further in their run and slows down progression proportionally as the run continues. Creates reward loops that trigger a sense of accomplishment in the player. Test method: Create a test environment with simulated level thresholds. Print the system outputs to the console and then observe how the amount of XP required for each level increases over time. Adjust the system if needed to ensure an appropriate amount of growth for each level threshold. Supporting context: Level systems in games usually hook players by making early levels extremely easy to achieve and then exponentially increasing the amount of effort needed to achieve later levels over time to incentivize the player to keep playing. Tracing information: (SYS-02) “There should be data persistence for neural network and player progression.”

	Priority: Could Have
16	SRID: OP-02 ID: NF-116 Title: Performance Across Operating Systems Statement: The game shall be able to achieve an average framerate of at least 60 FPS when running on Mac, Windows, or Linux. Rationale: 60 FPS is often cited as a good cutoff for smooth game performance. https://keyboardpollingratetest.com/what-is-good-fps-for-gaming-a-complete-guide-to-frame-rates/ We want our game to perform well across multiple operating systems, and framerate is one measure of performance. Test method: The game will be played for at least 5 minutes on an instantiation of each of the three listed operating systems. This will be done on a reasonably well-functioning device, such that it represents a typical example, even though in theory a slow device could cause a low framerate even for a performance-efficient game. Across the 5 minutes in each of the three cases, the game should achieve an average framerate of at least 60 FPS. Supporting context: Performance on computation tasks can vary across operating systems. https://www.makeuseof.com/i-benchmarked-my-gaming-pc-on-windows-vs-linux-and-the-difference-was-bigger-than-i-expected/ Tracing information: (SYS-07) “The game should perform smoothly on a typical device.” Priority: Should Have
17	SRID: OP-02 ID: NF-117 Title: GUI Across Operating Systems Statement: There shall be a fully functional and intuitive GUI across Mac, Windows and Linux instantiations of the game, without visual glitches. Rationale: The GUI is a crucial component of how the user interacts with the game, and so it should be fully functional and easy to use. And furthermore, it should be ensured that no problems or bugs appear in it in instantiations of the game running on different operating systems. Test method: We will form a checklist of important visual aspects of the game, and proceed to explore each aspect of the checklist on instantiations of the game on Mac, Windows, and Linux. Any visual glitches or unexpected difficulty in using the GUI will be

	noted. Supporting context: Each operating system tends to have a somewhat different approach to its visual display. https://tutoacademy.com/comparing-display-settings-across-different-operating-systems/ We want to ensure that our game's GUI works well across the setups that all three operating systems that we are focusing on tend towards having. Tracing information: (SYS-06) "There should be an intuitive and aesthetically fitting GUI, and a functional soundtrack." Priority: Must Have
18	SRID: CS-05 ID: NF-118 Title: State Machine Pattern Statement: The State Machine Pattern shall be templated to be reusable and maintainable across different entities. Rationale: The state machine pattern such as the one used for the player character controller will need to be templated in a reusable manner because it is a common design pattern that will need to be reused for many different entities across the project, especially enemies. Therefore, having a reusable pattern template will improve the maintainability of our project by providing a familiar style. Test method: Create several different types of entities with differing state machines using a templated design. If there are any issues with the template, work to fill in the gaps for future use. Supporting context: Almost every entity with more than one animation such as enemies and the player character will need a state machine to control their logic at different stages. Tracing information: (SYS-03) "There should be a fully realized combat system against enemies." Priority: Must Have
19	SRID: CC-03 ID: NF-119 Title: Rebindable Player Input Statement: The Rebinable Player Input shall allow users to rebind any input to any keyboard or controller key.

	Rationale: Allowing rebindable input makes the project more usable for a larger audience. If a control scheme we chose is uncomfortable to a user they can choose their own. Test method: Rebind all inputs with several different schemes that include conflicting bindings, and observe if the results are correct. Supporting context: A binding in this requirement means the keyboards keys or controller inputs associated with a specific action. Tracing information: (SYS-05) “There should be functional player controls, with an engaging character moveset.” Priority: Should Have
20	SRID: PG-06 ID: NF-120 Title: Procedural Generation Memory Footprint Statement: The memory footprint of the procedural generation process must operate within strict limits. Rationale: Limiting the memory footprint of the procedural generation process avoids excessive performance drops, such as lag or stuttering, and unnecessary computational strain on devices. Test method: Set up a test suite for the procedural generation system and observe the memory that it uses. Verify that it is discarding data that is out of the player’s immediate view to avoid using extra memory. Supporting context: Memory in this case refers to system RAM, used to hold active game data for the CPU to access quickly. Tracing information: (SYS-04) “There should be procedurally generated levels.” Priority: Should Have

Deliverables

Deliverable: A feature complete binary build of our game, Natural Selection.
Requirements: FR-101 Neural Network Control of Enemies, FR-102 Neural Net Knowledge Persistence, FR-103 Enemy Learning Through Neuroevolution, FR-104 Explicit Prediction of Psychological Traits, FR-105 Visibility of Enemy Strategy, FR-106 Player Movement Controller, FR-107 Player State Machine, FR-108 Melee Attack Collision System, FR-109 Projectile Attack System, FR-110 Soundtrack Player System, FR-111 Experience & Level Tracking System, FR-112 Player Data Persistence, FR-113 Procedural Generation Algorithms, FR-114 Level

Overview System, FR-115 Exponential Leveling Thresholds, NF-116 Performance Across Operating Systems , NF-117 GUI Across Operating Systems, NF-119 Rebindable Player Input

Deliverable: A documentation guide for developers working on the project that explains how subsystems work together.
Requirements: NF-118 State Machine Pattern

Standards

ISO 25010: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010
This standard defines a comprehensive quality model for software products, which is what our project is. This will provide clear metrics for us to evaluate the quality of our design and code and give us a structure to compare them against. We plan to ensure compliance with this standard by applying the 9 quality characteristics defined within our development process and final product evaluation.

IEEE 29148: https://standards.ieee.org/ieee/29148/6937/
This standard provides a comprehensive framework for engineering requirements across the software product lifecycle, something that will be used in our project. This will provide ways to construct good requirements to reduce ambiguity and scope creep, as well as set up clearer documentation . We plan to ensure compliance with this standard by adhering to the provisions mentioned in the standard to help create the guidelines that we will use throughout our software development process.

ISO 5055: https://www.iso.org/standard/80623.html
This standard defines source code quality measures to evaluate the internal quality of software systems before deployment, which is applicable to our project because our end goal is the deployment of our game to other platforms. This will impact our design and code because it provides four quality characteristics (security, reliability, performance efficiency, and maintainability) that are important to assess in any given software product. We plan to ensure compliance with the standard by measuring the quality of our design and code against those characteristics and determining what areas we need to work on for the highest possible quality end product.

GenAI statement

There was no generative AI usage in this assignment. No part of this assignment was written by GenAI.